Real-Time Weather ETL Pipeline

Insights Report: What happened, why it happened, and what should be done

Prepared by Zainab Danjuma | Week 7 Project, AnalystLab Africa Data Analytics Internship
Data snapshot: 8 August 2026, 07:43 | Cities: Lagos, Abuja, London

---

1. Executive Summary

This project built a working ETL (Extract, Transform, Load) pipeline in Python. It pulled live weather readings for **Lagos, Abuja and London** from the OpenWeather API, cleaned and structured them with Pandas, and saved the result to CSV for analysis. The pipeline ran end to end with **no missing values** across all 3 records and 6 fields.

The headline result: the two Nigerian cities were warmer and far more humid than London at the time of the snapshot. Lagos was the warmest and windiest city (24.67°C, 2.79 m/s), Abuja was the most humid (95%) under overcast skies, and London was the coolest, driest and calmest (17.57°C, 73%, 0.89 m/s).

Metric	Lagos	Abuja	London	Average
Temperature (°C)	**24.67** (highest)	22.57	17.57 (lowest)	21.60
Humidity (%)	89	**95** (highest)	73 (lowest)	85.67
Wind speed (m/s)	**2.79** (highest)	1.87	0.89 (lowest)	1.85
Condition	Broken Clouds	Overcast Clouds	Few Clouds	-

2. How the Pipeline Worked

The project followed the standard ETL structure:

Stage	What was done	Result
Extract	Looped through 3 cities and called the OpenWeather current-weather endpoint (metric units) with the Requests library.	3 JSON responses parsed into 3 records
Transform	Built a DataFrame, checked structure with info(), checked nulls with isnull().sum(), renamed columns to clean snake_case, converted date_time to a true datetime type.	6 clean columns, 0 nulls, correct data types
Load	Saved the cleaned DataFrame to weather_data.csv.	Analysis-ready CSV file
Analyse	Compared temperature, humidity, wind and conditions; found maximums; ran summary statistics; plotted three bar charts.	Findings in Section 3

Data quality was good for this small sample: all 18 values (3 rows x 6 columns) were present, numeric fields were stored as numbers, and the timestamp was converted to datetime64.

3. What Happened: The Findings

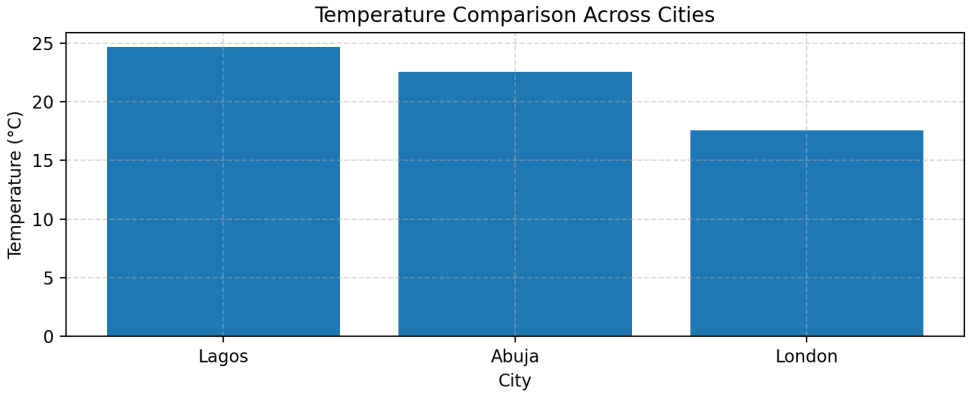


Figure 1. Temperature comparison across cities.

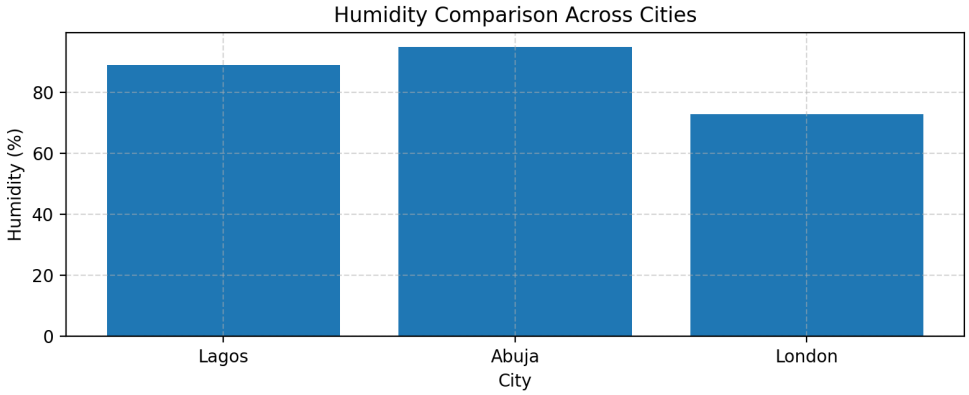


Figure 2. Humidity comparison across cities.

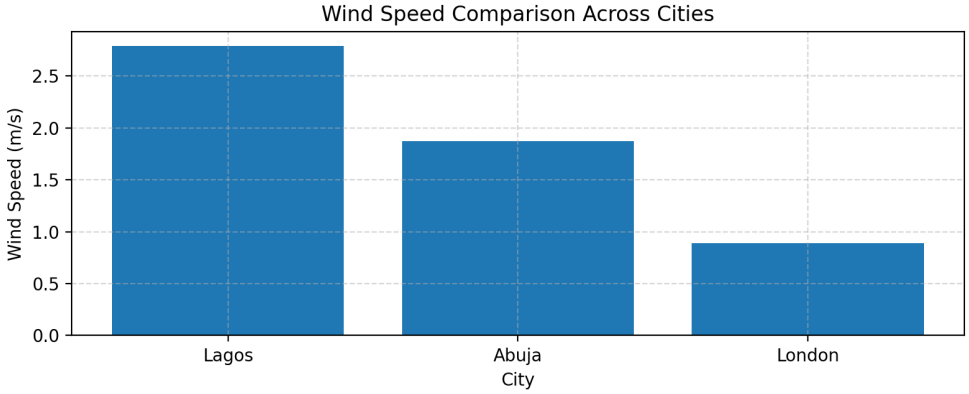


Figure 3. Wind speed comparison across cities (single snapshot, 8 August 2026).

- **Temperature:** Lagos was warmest at 24.67°C. London was 7.10°C cooler. The Nigerian cities averaged 23.62°C, about 6°C above London.
- **Humidity:** Abuja reached 95% and Lagos 89%, both very high. London sat at 73%, a 22-point gap below Abuja.
- **Wind:** All three cities had light winds (under 3 m/s). Lagos at 2.79 m/s was roughly 3.1 times London's 0.89 m/s.
- **Sky conditions:** Abuja was fully overcast, Lagos had broken clouds, and London had few clouds, so cloudiness tracked humidity closely across the three cities.
- **Spread:** Standard deviation was 3.65°C for temperature, 11.37 points for humidity and 0.95 m/s for wind, which shows the cities genuinely differ, mostly in humidity.

4. Why It Happened: Interpretation

The data shows *what* differed. The explanations below come from general climate knowledge and fit the pattern well, but with only one reading per city they are well-supported hypotheses, not proven causes.

Observation	Likely reason
Lagos is warmest and windiest	Lagos is a low-lying coastal city on the Atlantic. Warm ocean air keeps temperatures high, and sea-driven air movement from the coast gives it the strongest breeze of the three.
Abuja has the highest humidity (95%) with overcast skies	August falls in the middle of Nigeria's rainy season. Heavy cloud cover and moisture-laden air push relative humidity close to saturation, and the cloud blocks heat, keeping Abuja a little cooler than Lagos.
London is coolest, driest and calmest	London is at a much higher latitude with a temperate maritime climate. Few clouds and an early-morning reading (near the day's cooler hours) give lower temperature, lower humidity and very light wind.
Cloudiness and humidity move together	Cloud forms when air is humid, so the cloudier the sky (Abuja, then Lagos, then London), the higher the humidity.

Reading the numbers with care: the readings are one moment in time (07:43), so they show a morning snapshot, not a daily or seasonal average. Early-morning temperatures are usually near the daily low, which may understate each city's typical daytime warmth.

5. Pipeline Review: Issues Found in the Notebook

Reviewing the code and outputs turned up several points. Items marked High should be handled first.

Priority	Issue	Why it matters
High	The OpenWeather API key is written directly in the notebook code.	Anyone who sees the notebook or its GitHub repository can use and abuse the key. Keys exposed in public repos are routinely scraped.
Medium	No error handling: the loop assumes every request succeeds.	A bad city name, expired key or rate limit returns an error JSON, and the next line raises a KeyError that stops the whole run.
Medium	Timestamps use datetime.now() (the computer's clock), not the time of the weather observation.	The recorded time may not match when the weather was measured, and it is ambiguous across time zones.
Medium	to_csv() overwrites the file on each run.	Every run erases earlier data, so no history or trend can ever be built.
Low	Only 3 rows, one snapshot. describe() statistics (mean, std, quartiles) are not meaningful at n = 3.	Averages across 3 cities in different climates describe no real place, and cannot support trend claims.
Low	Units were dropped when columns were renamed (temperature, wind_speed). Some markdown headings have typos (e.g. "Cheack", "Wether", "Avearge").	Units in names (temperature_c) prevent misreading later; typos reduce polish in a portfolio piece.

6. Recommendations: What Should Be Done

A. Fix immediately

- **Revoke and regenerate the API key** in your OpenWeather account, because the current one has been written in plain text. Then load the new key from an environment variable or a .env file, and add .env to .gitignore.

B. Strengthen the pipeline

- **Add error handling:** check response.status_code, wrap each call in try/except, log failed cities and continue, and add a short retry.
- **Use the API's own observation time** (the dt field, converted to UTC, plus the timezone offset) instead of datetime.now().
- **Validate before loading:** confirm humidity is between 0 and 100, temperature is in a plausible range, and no city is duplicated.
- **Append, don't overwrite:** add each run to the CSV (or a SQLite/SQL table) so history accumulates. Put units in column names, e.g. temperature_c, wind_speed_ms.

C. Turn a snapshot into real insight

- **Schedule the pipeline** (Windows Task Scheduler, cron, or GitHub Actions) to run hourly or daily for at least 2 to 4 weeks.
- **Widen the scope:** add more cities (for example Port Harcourt, Kano, Accra, Nairobi) and more fields such as feels-like temperature, pressure, visibility and rainfall.
- **Analyse over time:** compare day versus night, rainy versus dry spells, and test whether humidity and cloud cover truly move together.
- **Build a Power BI or SQL dashboard** on top of the stored data, with city filters and trend lines, to show the full analyst workflow from raw API to decision-ready visual.

D. Practical use of the insights

- **Abuja (high humidity, overcast, rainy season):** outdoor operations, deliveries and events in August should plan for rain, and humidity-sensitive goods need dry storage.
- **Lagos (warmest, strongest breeze):** heat and hydration planning for outdoor work remains important despite the breeze.
- **Any weather-dependent business** (logistics, agriculture, retail, events) could use an automated feed like this as an early input for scheduling and risk alerts, once enough history is collected.

7. Conclusion

The pipeline did its core job: it extracted live data, produced a clean, complete dataset and revealed a clear pattern, with Lagos the warmest and windiest, Abuja the most humid and cloudy, and London the coolest and calmest. To move from a good learning project to a portfolio-grade one, secure the API key, harden the code, and collect data over time. That turns a single snapshot into genuine trend analysis.